

BGV/BFV Bootstrapping

SIMD 编码 / CoeffToSlot / digit extraction / idea 分享

梁朝阳

南开大学密码科学与技术

2026 年 4 月 18 日

- bootstrapping 在 BGV/BFV 中的整体流程?
- 什么是 SIMD 编码
- 具体算法以及优化

讲解主线

代数基础 → 系统级解密关系 → CoeffToSlot / digit extraction / SlotToCoeff → BSGS 与复杂度优化

原讲义的三段式结构

Part I

代数与数论基础

- 分圆环与分解型
- Frobenius 轨道
- CRT 与 slot 几何
- automorphism / rotation

Part II

主流程与表示变换

- BGV/BFV 解密关系
- sparse / fully packed
- CoeffToSlot 与 SlotToCoeff
- unpack / repack

Part III

优化与复杂度

- digit extraction
- weighted rotations
- BSGS 与 hoisting
- full / thin bootstrap

基本对象

在 power-of-two 情形，取 $m = 2N$ ，则

$$\Phi_m(X) = X^N + 1, \quad R_{p^e} = \mathbb{Z}_{p^e}[X]/(X^N + 1).$$

- 系数视角： $a(X) = a_0 + a_1X + \cdots + a_{N-1}X^{N-1}$ 。
- 分圆视角： $X^N + 1$ 不是随便挑的，而是第 m 个分圆多项式。
- **slot** 视角： CRT 之后，它会分解为若干个较小环的直积。

关键词

同一个明文环元素，可以在 *coefficient* 与 *slot* 两套坐标里来回解释。

从 Frobenius 轨道到 slot 个数

设 $d = \text{ord}_m(p)$, 即最小正整数使得 $p^d \equiv 1 \pmod{m}$ 。

核心推导链

$$d = \text{ord}_m(p) \implies \text{Frobenius 轨道长度} = d \implies \deg F_i = d \implies \ell = \frac{N}{d}.$$

- 在 $\mathbb{F}_p[X]$ 中, $\Phi_m(X)$ 分解成 $\ell = N/d$ 个不可约因子。
- Hensel 提升后, 在 $\mathbb{Z}_{p^e}[X]$ 中仍得到

$$X^N + 1 = F_1(X) \cdots F_\ell(X), \quad \deg F_i = d.$$

- 所以一个 ring element 同时像“长度 N 的系数向量”, 也像“ ℓ 个 slot, 每个 slot 内部宽度 d ”。

CRT 分解：为什么会出现 slot

中国剩余定理

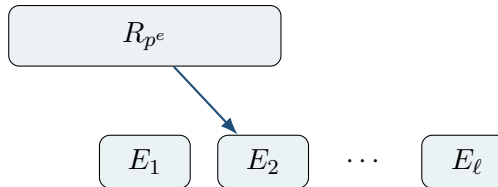
若 $F_1, \dots, F_\ell$ 两两互素，则

$$R_{p^e} \cong \prod_{i=1}^{\ell} \mathbb{Z}_{p^e}[X]/(F_i(X)) = E_1 \times \dots \times E_\ell.$$

- 每个 E_i 称为一个 **slot algebra**。
- 因为所有 E_i 彼此同构，常固定一个标准拷贝 E ，写成

$$R_{p^e} \cong E^\ell.$$

- 这就是后续 batching / SIMD 的数学来源。



分圆结构与参数链

核心参数关系

$$m = 2N, \quad \Phi_m(X) = X^N + 1, \quad d = \text{ord}_m(p), \quad \ell = \frac{N}{d}.$$

$$m = 2N \implies \Phi_m(X) = X^N + 1 \implies X^N + 1 = F_1(X) \cdots F_\ell(X)$$

$$\deg F_i = d \implies R_{p^e} \cong \prod_{i=1}^{\ell} E_i \cong E^\ell.$$

- N 是大环的系数维数。
- d 是 Frobenius 轨道长度，也是单个 slot 的内部宽度。
- ℓ 是 slot 数，决定外层 SIMD 规模。

Frobenius 轨道的具体写法

取一个 m 次本原单位根 ζ , 对任意 $h \in \mathbb{Z}_m^\times$, Frobenius 作用给出轨道

$$\zeta^h, \zeta^{hp}, \zeta^{hp^2}, \dots, \zeta^{hp^{d-1}}.$$

其中

$$d = \text{ord}_m(p) \iff p^d \equiv 1 \pmod{m}.$$

轨道长度恰为 d

$$\zeta^{hp^d} = \zeta^h, \quad \zeta^{hp^r} \neq \zeta^h \quad (0 < r < d).$$

因此每个不可约因子都可写成

$$F_h(X) = \prod_{t=0}^{d-1} (X - \zeta^{hp^t}).$$

slot 的内部维度是一个根在 Frobenius 作用下绕一圈所需的步数。

模 p 到模 p^e : 分解型如何提升

在 $\mathbb{F}_p[X]$ 中,

$$\Phi_m(X) = X^N + 1 = f_1(X) \cdots f_\ell(X), \quad \deg f_i = d.$$

若这些因子两两互素, 则由 Hensel 提升得到唯一的单首多项式

$$F_1(X), \dots, F_\ell(X) \in \mathbb{Z}_{p^e}[X]$$

满足

$$X^N + 1 = F_1(X) \cdots F_\ell(X), \quad F_i(X) \equiv f_i(X) \pmod{p}.$$

结论

$$\deg F_i = d, \quad R_{p^e} = \mathbb{Z}_{p^e}[X]/(X^N + 1) \cong \prod_{i=1}^{\ell} \mathbb{Z}_{p^e}[X]/(F_i(X)).$$

power-of-two 情形下的稀疏因子形状

对 $m = 2N$ 的 power-of-two cyclotomic, 原讲义强调每个 lifted factor 往往有稀疏形状:

$$F_i(X) = \begin{cases} X^d + b_i, & p \equiv 1 \pmod{4}, \\ X^d + a_i X^{d/2} + b_i, & p \equiv 3 \pmod{4}. \end{cases}$$

其中 $a_i, b_i \in \mathbb{Z}_{p^e}$ 。

这解释了为什么会自然出现步长

$$c = \begin{cases} d, & p \equiv 1 \pmod{4}, \\ d/2, & p \equiv 3 \pmod{4}, \end{cases} \quad Y = X^c, \quad R'_{p^e} \cong \mathbb{Z}_{p^e}[Y]/(Y^{N'} + 1).$$

意义

不是“为了 sparse packed 才去发明子环”，而是因子本身已经在提示：问题天然只依赖于 X^c 。

三套基底：系数基、局部幂基、统一幂基

系数基

大环 R_{p^e} 的自然基：

$$1, X, \dots, X^{N-1}.$$

局部幂基

第 i 个 slot 中：

$$1, \zeta_{m,i}, \dots, \zeta_{m,i}^{d-1},$$

统一幂基

固定标准拷贝 E ：

$$1, \zeta_m, \dots, \zeta_m^{d-1},$$

为什么 fully packed 里会出现 M

M 的工作是

$$\sum_{j=0}^{d-1} m_{i,j} \zeta_{m,i}^j \mapsto \sum_{j=0}^{d-1} m_{i,j} \zeta_m^j,$$

也就是把每个 slot 的局部坐标统一到同一个参考坐标系。

slot 几何、automorphism 与 rotation

- slot 标签来自商群 $\mathbb{Z}_m^\times / \langle p \rangle$ 。
- 由有限 Abel 群分解，可把 slot 编号写成多维循环坐标

$$h = g_1^{e_1} \cdots g_t^{e_t}, \quad 0 \leq e_i < \ell_i.$$

- 第 i 维 rotation 就是

$$(e_1, \dots, e_i, \dots, e_t) \mapsto (e_1, \dots, e_i + v, \dots, e_t).$$

automorphism

若 $j \in \mathbb{Z}_m^\times$ ，则

$$\tau_j : a(X) \mapsto a(X^j)$$

给出环自同构，并在 slot 视角下作用为

$$h \mapsto hj.$$

直觉

rotation 不是随意置换，而是底层分圆结构在 slot 索引群上的乘法作用。

sparse packed vs. fully packed

sparse packed

- 明文只依赖于 X^c
- 自然落入子环

$$R'_{p^e} \cong \mathbb{Z}_{p^e}[Y]/(Y^{N'} + 1), \quad Y = X^c$$

- slot 里通常装 **标量**
- U 本身就像 CRT/Vandermonde 矩阵

fully packed

- 每个 slot 里装的是 **d 维 E -元素**
- 有局部幂基与统一幂基之分
- 外层变换不只是 U ，而是

$$M^{-1}UM \quad \text{或} \quad M^{-1}U^{-1}M$$

- 为了做逐槽非线性，常常必须先 **unpack**

为什么会出现 Vandermonde / FFT-like 矩阵

在 sparse 子环中, 写

$$q(Y) = a_0 + a_1 Y + \cdots + a_{\ell-1} Y^{\ell-1}.$$

若 $y_i = \zeta_{m,i}^c$, 则其 slot 表示就是在这些点上的求值:

$$\begin{bmatrix} q(y_0) \\ q(y_1) \\ \vdots \\ q(y_{\ell-1}) \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & y_0 & y_0^2 & \cdots & y_0^{\ell-1} \\ 1 & y_1 & y_1^2 & \cdots & y_1^{\ell-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & y_{\ell-1} & y_{\ell-1}^2 & \cdots & y_{\ell-1}^{\ell-1} \end{bmatrix}}_U \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{\ell-1} \end{bmatrix}.$$

- 它既是 **Vandermonde**, 又长得像 **FFT**。
- sparse: 问题就是 U ; fully packed: 完整对象变成 $M^{-1}UM$ 。

密文本体始终是环元素短向量

最基本的 RLWE 密文形状是

$$c = (c_0, c_1) \in R_Q^2,$$

秘密钥为 $s \in R_Q$ ，解密线性部分通常长成

$$c_0 + c_1 s \quad \text{或更一般地} \quad \sum_i c_i s^i.$$

- slot 只是 **明文语义** 的解释坐标，不是密文对象类型。
- 因此 bootstrapping 的“同态解密”本质上是在 **密文域内模拟解密函数**。
- 噪声不可避免；问题不是消息没了，而是消息快被噪声淹没了。

BGV

$$u = c_0 + c_1 s \equiv m + p^e r \pmod{Q}$$

所以目标是

$$u \bmod p^e = m.$$

BFV

$$u = c_0 + c_1 s \equiv \Delta m + e \pmod{Q}$$

所以目标更像

$$m \approx \left\lfloor \frac{t}{Q} u \right\rfloor \pmod{t}.$$

和 CKKS 的差别

CKKS 的中心步骤更像近似 EvalMod；BGV/BFV 则是精确整数对象上的刷新。

BGV: 把解密关系显式写成 noisy plaintext

最核心的结构关系是

$$u := c_0 + c_1 s \equiv m + p^e r \pmod{Q}.$$

更一般地, 若密文有多个分量, 则

$$u := \sum_i c_i s^i \equiv m + p^e r \pmod{Q}.$$

对象含义

$$m \in R_{p^e}, \quad r \in R_Q, \quad u \in R_Q.$$

刷新目标

$$u \bmod p^e = m.$$

$$\text{Enc}(u) \rightsquigarrow \text{Enc}(u \bmod p^e) = \text{Enc}(m).$$

直观解释

BGV bootstrapping 的核心不是“重新算一个新消息”, 而是把 $m + p^e r$ 中的 p^e 倍垃圾同态剔除掉。

BFV: 核心更像 scale-and-round

BFV 中通常先把明文放大到大模数空间，抽象地写成

$$u := c_0 + c_1 s \equiv \Delta m + e \pmod{Q}, \quad \Delta \approx \frac{Q}{t}, \quad t = p^e.$$

因此消息恢复更像

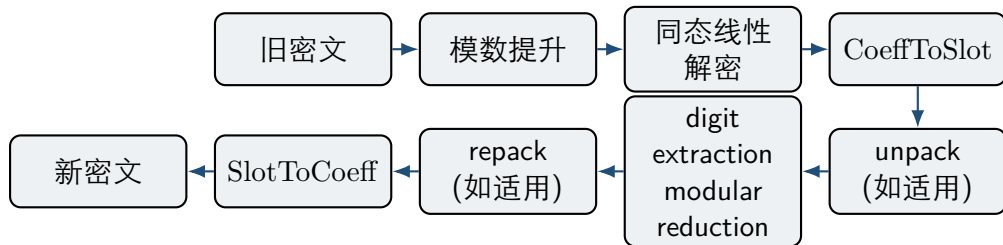
$$m \approx \left\lfloor \frac{t}{Q} u \right\rfloor \pmod{t}.$$

与 BGV 的并列比较

$$\text{BGV: } m + p^e r \xrightarrow{\text{mod } p^e} m, \quad \text{BFV: } \Delta m + e \xrightarrow{\text{scale+round}} m.$$

- BGV 更像精确保留低位 residue。
- BFV 更像把被放大的消息缩回原范围并正确取整。

BGV bootstrapping 的完整流程



一句话概括

先把问题显式化为 $u = m + p^e r$ ，再切到适合逐槽处理的表示，做“去掉 p^e 倍垃圾”的非线性清理，最后重组回合法 ring ciphertext。

每一步在做什么

- ① 模数提升：不是刷新噪声，只是为后续同态计算腾出空间。
- ② 同态线性解密：把密文变成显式 noisy plaintext，如 $u = m + p^e r$ 。
- ③ CoeffToSlot：把“环上的系数表示”转成“适合逐槽处理的表示”。
- ④ **unpack**：若是 fully packed，需要把 E -slot 拆成标量槽流。
- ⑤ **digit extraction**：真正的刷新核心，逐槽恢复 residue / 低位信息。
- ⑥ **repack** + SlotToCoeff：把处理后的槽重新装配回标准 ring ciphertext 形状。

最容易混淆的点

“线性解密”并没有清噪；真正让噪声预算回来的，是后面的非线性步骤。

CoeffToSlot / SlotToCoeff 的角色

CoeffToSlot

把大环中的系数信息搬到 slot 视角下，目的是让后续非线性变成

$$(u_1, \dots, u_\ell) \mapsto (f(u_1), \dots, f(u_\ell)).$$

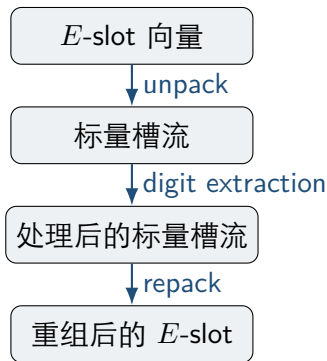
- sparse packed: 主要是一个 U (及其逆) 的问题。
- fully packed: 要处理局部幂基与统一幂基, 故完整变换长成 $M^{-1}UM$ 。
- 这一步本身虽然线性, 但通常是 bootstrapping 中的大头之一。

SlotToCoeff

把处理后的 slot 数据重新组合回标准多项式 / ring 表示, 保证输出仍是合法密文。

为什么 fully packed 必须 unpack / repack

- 在 fully packed 情况下，单个 slot 里装的不是标量，而是 E -元素。
- 但 digit extraction 往往要求输入已经是 **标量槽**。
- 因此必须先 **unpack**：把每个 E -slot 展开成可逐槽处理的标量流。
- 非线性处理完成后，再通过 **repack** 逆向重组回 fully packed 结构。



digit extraction: 刷新噪声的核心

BGV 的本质任务

对每个槽，输入是

$$u_i = m_i + p^e r_i,$$

希望恢复

$$m_i.$$

- 典型思路：base- p 逐位提取、base- p^k 分块、carry/correction 视角。
- CRT 之后可逐槽处理，因此非常适合 SIMD 化。
- 但它的本质是 **非线性整数函数**，所以深度与成本都高。

对比

BGV: 精确 residue 恢复; BFV: 更偏 scale-and-round; CKKS: 更偏近似 EvalMod。

sparse packed 下的 CoeffToSlot: 本质是求值

在 sparse 子环中写

$$q(Y) = a_0 + a_1 Y + \cdots + a_{\ell-1} Y^{\ell-1}, \quad Y = X^c.$$

若 $y_i = \zeta_{m,i}^c$, 则

$$\text{CoeffToSlot}(q) = \begin{bmatrix} q(y_0) \\ q(y_1) \\ \vdots \\ q(y_{\ell-1}) \end{bmatrix} = U \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{\ell-1} \end{bmatrix}, \quad U_{ik} = y_i^k = \zeta_{m,i}^{ck}.$$

解释

CoeffToSlot 就是把系数向量 $(a_0, \dots, a_{\ell-1})$ 送到求值向量

$$(q(y_0), q(y_1), \dots, q(y_{\ell-1})).$$

sparse packed 下的 SlotToCoeff: 本质是插值

已知所有 slot 值后, 反向变换就是插值:

$$\text{SlotToCoeff} = U^{-1}, \quad \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{\ell-1} \end{bmatrix} = U^{-1} \begin{bmatrix} q(y_0) \\ q(y_1) \\ \vdots \\ q(y_{\ell-1}) \end{bmatrix}.$$

因此 CoeffToSlot / SlotToCoeff 在 sparse packed 下就是一对求值 / 插值变换。

$$q(Y) \xrightarrow{\text{CoeffToSlot}} (q(y_0), \dots, q(y_{\ell-1})) \xrightarrow{\text{SlotToCoeff}} q(Y).$$

重点

这也是为什么它看起来既像 Vandermonde 变换, 又像 FFT: 节点本身来自 roots of unity 的幂。

fully packed 下为什么不是单独一个 U

设第 k 个 slot 的消息写成局部幂基形式

$$m_k = \sum_{j=0}^{d-1} m_{k,j} \zeta_{m,k}^j.$$

先用 M 统一成标准幂基:

$$M : \sum_{j=0}^{d-1} m_{k,j} \zeta_{m,k}^j \mapsto v_k = \sum_{j=0}^{d-1} m_{k,j} \zeta_m^j.$$

然后才有资格做外层 slot 混合:

$$(v_1, \dots, v_\ell)^T \xrightarrow{U} (w_1, \dots, w_\ell)^T.$$

最后再送回各自局部基:

$$M^{-1} : w_k \mapsto \sum_{j=0}^{d-1} \tilde{m}_{k,j} \zeta_{m,k}^j.$$

unpack: 把 E -slot 展开成标量槽流

fully packed 时, 一个 slot 里不是标量, 而是

$$m_i = m_{i,0} + m_{i,1}\zeta_{m,i} + \cdots + m_{i,d-1}\zeta_{m,i}^{d-1}.$$

于是

$$(m_1, \dots, m_\ell)$$

要被展开成若干条标量流, 例如

$$(m_{1,0}, \dots, m_{\ell,0}), (m_{1,1}, \dots, m_{\ell,1}), \dots, (m_{1,d-1}, \dots, m_{\ell,d-1}).$$

目的

把原本活在 E 里的对象, 变成 digit extraction 可以逐槽处理的标量对象。

repack: 把处理后的标量流重新装回去

当每条标量流都完成非线性清理后，再逆向恢复每个 slot:

$$m_i = \sum_{j=0}^{d-1} m_{i,j} \zeta_{m,i}^j.$$

于是

$$(m_{1,0}, \dots, m_{\ell,0}), \dots, (m_{1,d-1}, \dots, m_{\ell,d-1})$$

被重新组装为

$$(m_1, \dots, m_{\ell}).$$

记忆方式

unpack 是“拆成坐标流”，repack 是“按局部幂基重组回 slot 元素”。

digit extraction 的递归形式

若要恢复 $u \bmod p^e$ ，一种最直接的写法是 base- p 逐位提取。先写

$$u \bmod p^e = d_0 + d_1p + \cdots + d_{e-1}p^{e-1}.$$

递归定义

$$u^{(0)} = u, \quad d_t = u^{(t)} \bmod p, \quad u^{(t+1)} = \frac{u^{(t)} - d_t}{p}.$$

于是最终

$$u \bmod p^e = d_0 + d_1p + \cdots + d_{e-1}p^{e-1}.$$

- 优点：概念最直观。
- 代价：需要做 e 轮，深度会累积。

digit extraction 的 toy 例子：低位保留，高位丢弃

取一个槽值

$$u = 68.$$

在 base-3 中，

$$68 = 2 \cdot 3^0 + 1 \cdot 3^1 + 1 \cdot 3^2 + 2 \cdot 3^3 = (2112)_3.$$

若当前目标是恢复 $u \bmod 9$ ，则只保留最低两位：

$$(12)_3 = 1 \cdot 3 + 2 = 5, \quad 68 \bmod 9 = 5.$$

同样地，从 base-9 视角看

$$68 = 5 + 9 \cdot 7,$$

所以最低 block 也是 5。

一句话

BGV 的 digit extraction 就是：保留低位（或低块），丢掉高位（或高块）。

为什么线性变换也是 bootstrapping 的大头

同态世界里的“矩阵乘法”并不直接

普通数值计算里的 $y = Ax$ ，在 FHE 中必须拆成：

- rotation / automorphism,
- plaintext mask multiplication,
- 多项求和,
- 每次 automorphism 后的 key switching.

真实成本

为了实现“一个矩阵乘法”，关键不是矩阵有多大，而是需要多少次 automorphism、key switching 和明文乘法。

weighted rotations 与 BSGS

weighted rotations

很多大线性变换都可写成

$$y = \sum_k D_k \rho_k(x),$$

其中 ρ_k 是 rotation, D_k 是明文对角掩码。

BSGS

Baby-step/Giant-step 的目标是：

- 减少 rotation 种类数；
- 重用中间结果；
- 降低 key switching 总量。

- one-dimensional / multidimensional BSGS 都来自 slot 索引群的坐标结构。
- good dimension / bad dimension 会直接影响能否 “一个 rotation 对应一个 automorphism”。

hoisting、FFT-like 分解与矩阵结构优化

- **hoisting / double-hoisting**: 提前共享某些分解或 key switching 代价, 减少重复工作。
- **U 矩阵分解**: 把看似稠密的变换拆成若干稀疏矩阵乘积或蝶形结构。
- **好处**:
 - 减少 automorphism 数量;
 - 改善缓存 / 预处理复用;
 - 把“像 FFT 但不完全一样”的结构真正落到同态实现里。

一句话

digit extraction 决定了 非线性深度, 而 BSGS + FFT-like 分解决定了 线性层的工程可行性。

weighted rotations 的矩阵公式

设

$$x = (x_0, x_1, \dots, x_{n-1})^T, \quad y = Ax, \quad A \in M_n(F).$$

记 P_k 为循环移位矩阵, 则任意线性变换都可唯一写成

$$A = \sum_{k=0}^{n-1} D_k P_k,$$

其中每个 D_k 都是对角矩阵。因此

$$y = Ax = \sum_{k=0}^{n-1} D_k (P_k x).$$

同态实现翻译

$$P_k x \rightsquigarrow \rho_k(x), \quad D_k \rightsquigarrow \text{plaintext mask multiplication}.$$

所以一整个矩阵乘法会被翻译成 “很多次 rotation + 明文乘 + 求和”。

BSGS: 把很多次 rotation 重新组织

若直接实现

$$y = \sum_{k=0}^{n-1} D_k \rho_k(x),$$

rotation 种类数通常是 $O(n)$ 。BSGS 的做法是把索引拆成

$$k = i + jg, \quad 0 \leq i < g, \quad 0 \leq j < \left\lceil \frac{n}{g} \right\rceil.$$

于是

$$y = \sum_j \rho_{jg} \left(\sum_i D_{i+jg} \rho_i(x) \right).$$

选择 $g \approx \sqrt{n}$ 时, rotation 组织规模从

$$O(n) \quad \text{下降到} \quad O(\sqrt{n})$$

这一量级 (再配合 hoisting, 可进一步降低有效 key switching 成本)。

复杂度与参数：先看结构参数

最关键的两个量

$$d = \text{ord}_m(p), \quad \ell = \frac{N}{d}.$$

- d 小：slot 多，内部浅，fully packed 更容易处理。
- d 大：slot 少，但内部代数结构更厚， M/M^{-1} 、unpack/repack 成本更高。
- ℓ 大：外层线性层更大，rotation 组织规模上升。

复杂度与参数：再看成本指标

至少同时看四类指标

depth, #rot, #pt-ct mult, #KS.

- digit extraction 主导非线性深度；
- CoeffToSlot/SlotToCoeff 与 unpack/repack 常主导 rotation / key switching 成本；
- fully packed 与 sparse packed 的差距，往往是结构性差异，不只是常数倍差异。

full bootstrap 还是 thin bootstrap

full bootstrap

- 输出仍保持 fully packed
- 吞吐高
- 需要完整的
CoeffToSlot、unpack / repack、
SlotToCoeff
- 通常代价更高

thin bootstrap

- 输出压缩到更小子环或更少自由度表示
- 结构更简单
- 代价更低
- 后续并行度与 packing 能力下降

选择逻辑

取决于应用是否极度依赖满吞吐 SIMD、当前参数下 unpack/repack 是否太贵、以及是否接受用更多密文分摊后续计算。

复杂度：真正贵的到底是什么

- ① digit extraction 的非线性深度；
- ② CoeffToSlot / SlotToCoeff 与 unpack / repack 里的大线性变换；
- ③ automorphism + key switching 的累计次数；
- ④ fully packed 相对 sparse packed 的额外内部基变换成本；
- ⑤ 参数选择对 slot 数、内部维数、模数链长度的联动影响。

经验判断

若只看“数学步骤数”，常常会低估线性层的真实代价；实现时往往是 **线性层 + key switching** 和 **digit extraction** 一起构成主瓶颈。

Idea: 把已有大环密文在线转成“族表示”

实际上最初的设想就是拆明文-> 通过空间换可并行性与复杂度降低。

想回答的问题

如果存在一个黑盒模块压缩原语 `CompactStep`，那么能否把

已有的大环普通密文

在线转换成

结构化的小环密文族

并在族表示上继续做同态加法、乘法，最后再解密与聚合？

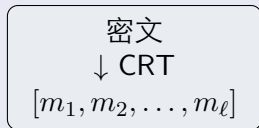
这个思想我想到后去搜索了一些论文，有一篇 2026 年的预印本：High-Precision Exact FHE Made Simple, General, and Fast。他的思想和我类似也是拆明文，只是拆法不是特别一样。在某种意义上说明拆明文这件事是有意义的。（也有别人在干这件事情）

有一个想说明的是这个拆明文和 SIMD 编码把明文融到一个密文里面是否矛盾。

拆明文 vs. SIMD 编码：矛盾吗？

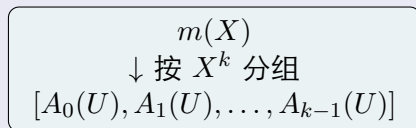
SIMD 编码 (Batching)

- 把 **多个独立明文** 打包进 **一个密文**
- 利用环的 CRT 分解，每个 slot 存一个明文
- 目标：并行吞吐——一个密文算多件事



拆明文 (族转换)

- 把 **一个大明文** (长多项式) 拆成 **多个小环密文**
- 每个小密文存一个剩余类分量 $A_j(U)$
- 目标：结构降维——降低旋转深度、KeySwitching 成本



为什么不矛盾？

- ① 层次不同：SIMD 是“空间并行”，拆明文是“结构分解”，可叠加使用。

黑盒原语 CompactStep: 单层减半压缩

输入

父层环

$$R_{2n} = \mathbb{Z}_q[X]/(X^{2n} + 1)$$

上的两项密文

$$ct = (a(X), b(X)),$$

满足

$$b(X) - a(X)s(X) = m(X) + e(X).$$

输出

两个子层环

$$R_n = \mathbb{Z}_q[Y]/(Y^n + 1), \quad Y = X^2$$

上的两项密文 $\tilde{ct}_0, \tilde{ct}_1$, 分别解密为

$$m_0(Y) + \tilde{e}_0(Y), \quad m_1(Y) + \tilde{e}_1(Y),$$

其中

$$m(X) = m_0(X^2) + X m_1(X^2).$$

- 正确性：输出确实对应偶/奇子问题；
- 维度下降：宿主环从 R_{2n} 真的降到 R_n ；
- 噪声可控：若父层噪声为 η , 则子层噪声至多为 $\Gamma_n(\eta)$ ；
- 单次成本记为 $K_{\text{mc}}(n)$ 。

从大环密文到族表示：递归 q 层

设

$$N = 2^L, \quad k = 2^q, \quad n = \frac{N}{k}.$$

任意大明文

$$m(X) \in \mathbb{Z}_t[X]/(X^N + 1)$$

都可唯一写成按模 k 剩余类分组的形式

$$m(X) = \sum_{j=0}^{k-1} X^j A_j(U), \quad U = X^k,$$

其中

$$A_j(U) \in \mathbb{Z}_t[U]/(U^n + 1).$$

族表示的语义

把消息表示成

$$C = (ct_0, \dots, ct_{k-1}),$$

其中每个 ct_j 都是小环

$$S_n = \mathbb{Z}_q[U]/(U^n + 1)$$

上的普通两项密文，并加密 $A_j(U)$ 。

在线转换链条与入口成本

在线转换结果

经过 $q = \log_2 k$ 层递归压缩后,

$$ct^{(0)} \implies C = (ct_0, \dots, ct_{k-1}),$$

其中每个 ct_j 都活在小环

$$S_n = \mathbb{Z}_q[U]/(U^n + 1), \quad U = X^k,$$

并加密对应的剩余多项式 $A_j(U)$ 。

总转换代价

第 t 层有 2^t 个节点，每个节点规模为 $N/2^{t+1}$ ，因此

$$T_{\text{to_family}}(N, k) = \sum_{t=0}^{q-1} 2^t K_{\text{mc}}\left(\frac{N}{2^{t+1}}\right).$$

若黑盒代价满足

$$K_{\text{mc}}(n) = O(n),$$

则有

$$T_{\text{to_family}}(N, k) = O(N \log k).$$

这个 $O(N \log k)$ 是哪一步的收益？

它是“**在线把已有大环密文转成结构化族表示**”的入口成本，不是说进入族表示后任意乘法都自动变快。

族表示上的同态运算：加法简单，乘法是卷积

设

$$m(X) = \sum_{i=0}^{k-1} X^i A_i(U), \quad n(X) = \sum_{j=0}^{k-1} X^j B_j(U).$$

族加法：逐分量相加

$$(C \oplus D)_j = ct_j \oplus dt_j,$$

明文上对应

$$A_j(U) + B_j(U).$$

若第 j 分量噪声分别为 $\nu_{A,j}, \nu_{B,j}$ ，则

$$\nu_{+,j} = \nu_{A,j} + \nu_{B,j}.$$

族乘法：不是逐分量乘，而是剩余类卷积

乘积可写成

$$m(X)n(X) = \sum_{r=0}^{k-1} X^r C_r(U),$$

其中

$$C_r(U) = \sum_{i+j=r} A_i(U)B_j(U) + U \sum_{i+j=r+k} A_i(U)B_j(U).$$

因此密文算法应先对每一对 (i, j) 做小环乘法，得到 $\mu_{i,j}$ ，再聚合成

$$\tilde{ct}_r = \sum_{i+j=r} \mu_{i,j} + U \sum_{i+j=r+k} \mu_{i,j}.$$

在线大环转族：为什么入口成本是 $O(N \log k)$ (1/2)

递归 q 层压缩

设

$$k = 2^q, \quad n = N/k.$$

从一个规模为 N 的普通大环密文出发，每做一层 CompactStep,

- 节点数翻倍；
- 每个子问题的环规模减半。

因此第 t 层（从 $t = 0$ 开始）有

$$2^t$$

个节点，而每个节点调用黑盒的规模是

$$N/2^{t+1}.$$

总转换代价

若单次模块压缩代价记为 $K_{\text{mc}}(n)$, 则

$$T_{\text{to_family}}(N, k) = \sum_{t=0}^{q-1} 2^t K_{\text{mc}}\left(\frac{N}{2^{t+1}}\right).$$

特别地, 若黑盒满足线性代价

$$K_{\text{mc}}(n) = O(n),$$

则第 t 层总代价为

$$2^t \cdot O\left(\frac{N}{2^{t+1}}\right) = O(N),$$

一共 $q = \log_2 k$ 层, 所以

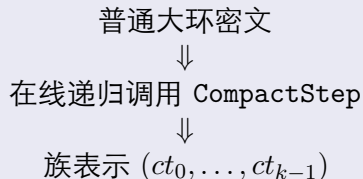
$$T_{\text{to_family}}(N, k) = O(N \log k).$$

在线大环转族：为什么入口成本是 $O(N \log k)$ (2/2)

这条复杂度对应哪一步？

$$O(N \log k)$$

对应的是：



也就是 “在线接入结构化表示”的入口成本。

结论与下一步

- 这条链条把一个“假设存在的压缩原语”提升成了一个完整系统级框架：

普通大环密文 $\rightarrow$ 在线族转换 $\rightarrow$ 族上同态计算 $\rightarrow$ 解密与聚合.

- 它最适合回答的问题不是“通用 FHE 是否更快”，而是：

- 如何对已有密文做在线结构化接入；
- 如何支持部分输出与部分递归任务。

- 因而两条路线的定位应当分开看：

静态族方案 $\Rightarrow$ 可计算性， 在线压缩方案 $\Rightarrow$ 在线接入性.